
REPORT & EXPERIMENT: AUTOMATIC CHANGE-POINT DETECTION IN TIME SERIES VIA DEEP LEARNING

PAPER REPORT

 **Jingzhi Sun**^{*}
Department of Statistics
University of California, Davis
Davis, CA 95618
edsun@ucdavis.edu

 **Yancheng Zhou**[†]
Department of Statistics
University of California, Davis
Davis, CA 95618
@ucdavis.edu

June 7, 2026

ABSTRACT

This paper studies a deep learning framework for offline change-point detection that reinterprets classical methods such as CUSUM and likelihood-ratio tests as neural network classifiers. Under this perspective, change-point detection can be formulated as a supervised classification problem trained on labeled time series data. The paper establishes theoretical bounds on generalization error and shows that neural network-based methods achieve performance comparable to classical procedures under ideal assumptions and improved robustness under model misspecification.

Our project will review the theoretical connections between classical change-point detection and neural network representations, reproduce selected simulation results, and investigate whether deeper architectures exhibit implicit regularization effects that stabilize generalization performance beyond what classical VC-dimension theory predicts.

Keywords Time Series · Deep Learning · Change-Point Detection

1 Introduction

Change-point detection is a fundamental problem in time series analysis, with wide applications in areas such as signal processing, finance, and environmental studies. The goal is to identify whether and when the underlying data-generating mechanism changes over time. Classical statistical methods, such as the CUSUM (Cumulative Sum) test, are widely used due to their simplicity, interpretability, and strong theoretical guarantees under ideal assumptions. However, these methods often rely on restrictive conditions, such as independence or specific noise distributions, and their performance may degrade under model misspecification or complex dependence structures.

In recent years, machine learning approaches, particularly neural networks, have emerged as flexible alternatives that can adapt to more complex data patterns. Despite their empirical success, their theoretical properties in change-point detection are not yet fully understood.

This report studies a recent paper that connects classical change-point detection methods with neural network models. Our goal is to clarify the main ideas of the paper, understand its theoretical results, and reproduce its empirical findings.

In Section 2, we begin by reviewing the classical change-point detection problem in time series. We introduce the standard change-in-mean model and explain the principle of the CUSUM (Cumulative Sum) classifier, which detects structural changes by comparing segment means. We then discuss how this classical procedure can be represented within a neural network hypothesis class, showing that the CUSUM classifier is in fact a special case of a shallow neural network. This provides a bridge between traditional statistical methods and modern learning-based approaches, and helps explain why neural networks can achieve strong performance in this problem.

^{*}First year Ph.D student in UC Davis

[†]First year M.S student in UC Davis

Next, we organize and explain the generalization error analysis presented in the paper. The key idea is to use the CUSUM classifier as a benchmark to control the approximation error, while the estimation error is bounded using VC dimension theory. This leads to a decomposition of the test error into two parts: one inherited from the statistical performance of CUSUM, and the other determined by the complexity of the neural network class.

We then describe how the authors extend these results in two directions: first, from single-layer networks to deeper architectures, and second, from using the full time series to using only a subset of candidate change-point locations. These extensions highlight how network depth and input reduction affect both the approximation and complexity terms in the generalization bound.

In addition to the theoretical analysis, we reproduce the simulation studies from the original paper. The experiments consider a binary classification setting where the goal is to detect the presence of a change-point under different noise structures, including independent Gaussian noise, autoregressive dependence, random correlation, and heavy-tailed (Cauchy) noise. The neural network models include both shallow networks that can exactly represent the CUSUM procedure and deeper, overparameterized architectures.

Our reproduction results partially differ from those reported in the original paper. In particular, the CUSUM baseline often performs stronger than the neural networks in our implementation, and some neural network configurations exhibit poor generalization performance. We further investigate the effects of network width, depth, label noise, and optimization hyperparameters, aiming to understand whether implicit regularization plays a role in this setting.

Overall, this report provides a structured interpretation of the paper, combining theoretical clarification with empirical investigation, and highlights both the strengths and the limitations of neural network approaches to change-point detection.

2 Main Idea

2.1 Model of Change Point and CUSUM-Based Classifier

We first introduce the model of simple change in Mean.

Simple Change in Mean

$$\begin{aligned} X &= \mu + \xi, \\ \mu &= (\mu_L \mathbf{1}\{i \leq \tau\} + \mu_R \mathbf{1}\{i > \tau\})_{i \in [n]} \in \mathbb{R}^n. \end{aligned} \tag{1}$$

where X , μ is $n \times 1$. μ_L, μ_R are the unknown signal values before and after the change-point. $\xi \in N_n(0, I_n)$.

The traditional statistical method to detect whether there exists a change point τ in the mean is the **Cumulative Sum(CUSUM)**.

The CUSUM procedure examines each potential time point by comparing the mean of the observations before that point with the mean of the observations after it. If there is a significant difference between the two segments, it suggests the presence of a change in the mean.

First, we construct CUSUM transformation $C : \mathbb{R}^n \rightarrow \mathbb{R}^{n-1}$.

$$C(X) = (v_1^T X, v_2^T X, \dots, v_{n-1}^T X)$$

$$\text{where } v_i := \left(\sqrt{\frac{n-i}{in}} \mathbf{1}_i^\top, -\sqrt{\frac{i}{(n-i)n}} \mathbf{1}_{n-i}^\top \right)^\top.$$

As a result, $v_i^T X = \sqrt{\frac{i(n-i)}{n}} (\bar{X}_{1:i} - \bar{X}_{i+1:n})$ and $\text{var}(v_i^T X) = 1$. Then we can give the final statistics h_λ^{CUSUM} as

$$h_\lambda^{\text{CUSUM}}(X) = 1(\|C(X)\|_\infty > \lambda)$$

If the maximum difference in mean is larger than λ , then there is a change point. CUSUM Statistics depend on $\max(v_i^T x)$.

Generalization of Simple Change in Mean Model

Here, we extend the change-point model to a regression setting with dependent noise:

$$X = Z\beta + C_\tau \phi + \Gamma \xi, \tag{2}$$

where $Z \in \mathbb{R}^{n \times p}$ is the design matrix, $\beta \in \mathbb{R}^p$ is the regression coefficient, C_τ represents the change-point structure at location τ , ϕ denotes the magnitude of the change, and $\Gamma\xi$ models dependent noise.

The classical change-in-mean problem can be viewed as a special case of the model above. We take $p = 1$ and let Z be a column vector of ones. The vector c_τ is defined such that its first τ entries are zero and the remaining entries are one. Under this formulation, β represents the mean before the change point, while ϕ denotes the magnitude of the change.

More explicitly,

$$X = \mathbf{1}_n \beta + (\underbrace{0, \dots, 0}_\tau, \underbrace{1, \dots, 1}_{n-\tau})^\top \phi + \Gamma\xi.$$

Also, the change in slope problem can be expressed as

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ \vdots & \vdots \\ 1 & n \end{pmatrix} \beta + \begin{pmatrix} 0 \\ \vdots \\ \max(0, i - \tau) \\ \vdots \\ n - \tau \end{pmatrix} \phi + \Gamma\xi.$$

After the change point the slope becomes from β_1 to $\beta_1 + \phi$. To test whether there is a change point in slope, it actually needs to test $H_0 : \phi = 0$ v.s $\phi \neq 0$.

2.2 Mathematical foundation of Machine learning Methods

This section establishes the central theoretical connection of the paper: classical CUSUM-type change-point tests and their generalisations can be represented as neural networks. The purpose of this result is to show that widely used statistical procedures are special cases of neural network classifiers, thereby embedding traditional change-point detection within a broader learning framework.

Lemma 1. For any $\lambda > 0$, we have $h_\lambda^{\text{CUSUM}} \in \mathcal{H}_{1, 2n-2}$.

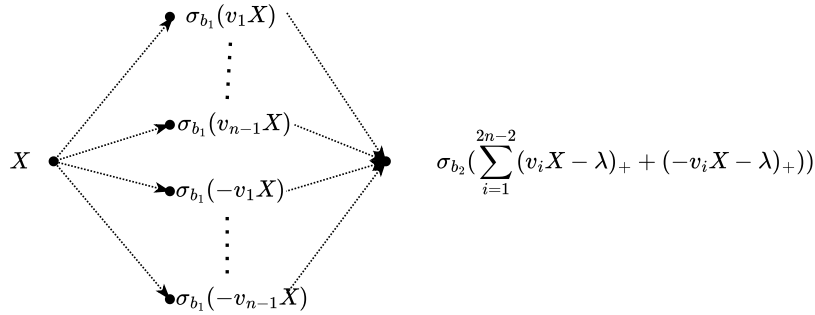


Figure 1: Diagram of Neural Network framework

Proof. Define

$$W_0 := (v_1, \dots, v_{n-1}, -v_1, \dots, -v_{n-1})^\top$$

and

$$W_1 := \mathbf{1}_{2n-2}, \quad b_1 := \lambda \mathbf{1}_{2n-2}, \quad b_2 := 0.$$

Then

$$h(x) := \sigma_{b_2}^* W_1 \sigma_{b_1} W_0 x \in \mathcal{H}_{1, 2n-2}$$

can be rewritten as

$$h(x) = \mathbf{1} \left\{ \sum_{i=1}^{n-1} [(v_i^\top x - \lambda)_+ + (-v_i^\top x - \lambda)_+] > b_2 \right\} = \mathbf{1} \{ \|C(x)\|_\infty > \lambda \} = h_\lambda^{\text{CUSUM}}(x),$$

as desired. $\square$

Also we can extend to the regression form 2

Lemma 2. *Consider the change-point model 2 with a possible change at $\tau \in [n-1]$. Assume further that Γ is invertible. Then there exists an $h^* \in \mathcal{H}_{1, 2n-2}$ equivalent to the likelihood-ratio test for testing $\phi = 0$ against $\phi \neq 0$.*

Sketch of proof. To test

$$H_0 : \phi = 0 \quad \text{against} \quad H_1 : \phi \neq 0,$$

we consider the likelihood-ratio test for the change-point model

$$X = Z\beta + c_\tau\phi + \Gamma\xi, \quad \xi \sim N(0, I_n).$$

Since Γ is invertible, we may left-multiply the model by Γ^{-1} and obtain the equivalent model

$$\tilde{X} = \tilde{Z}\beta + \tilde{c}_\tau\phi + \xi,$$

where

$$\tilde{X} := \Gamma^{-1}X, \quad \tilde{Z} := \Gamma^{-1}Z, \quad \tilde{c}_\tau := \Gamma^{-1}c_\tau.$$

Thus the noise is now i.i.d. standard Gaussian, so the likelihood-ratio test is equivalent to the usual Gaussian F -test for testing whether the additional regressor $\tilde{c}_\tau$ contributes to the linear model.

Now decompose $\tilde{c}_\tau$ into its component in the column space of $\tilde{Z}$ and its orthogonal complement:

$$\tilde{c}_\tau = P_{\tilde{Z}}\tilde{c}_\tau + (I - P_{\tilde{Z}})\tilde{c}_\tau,$$

where $P_{\tilde{Z}}$ denotes the orthogonal projection onto $\text{col}(\tilde{Z})$. The component $P_{\tilde{Z}}\tilde{c}_\tau$ can be absorbed into the nuisance term $\tilde{Z}\beta$, and therefore does not affect the testing problem. Hence, without loss of generality, we may replace $\tilde{c}_\tau$ by

$$(I - P_{\tilde{Z}})\tilde{c}_\tau$$

and assume that

$$\tilde{c}_\tau \perp \text{col}(\tilde{Z}).$$

Under this orthogonality condition, the reduction in residual sum of squares from adding $\tilde{c}_\tau$ to the null model is

$$\tilde{X}^\top \tilde{c}_\tau (\tilde{c}_\tau^\top \tilde{c}_\tau)^{-1} \tilde{c}_\tau^\top \tilde{X}.$$

This is a rank-one quadratic form. Indeed, if we define

$$v_\tau := \frac{1}{\sqrt{\tilde{c}_\tau^\top \tilde{c}_\tau}} \tilde{c}_\tau^\top \Gamma^{-1},$$

then

$$\tilde{X}^\top \tilde{c}_\tau (\tilde{c}_\tau^\top \tilde{c}_\tau)^{-1} \tilde{c}_\tau^\top \tilde{X} = \left(\frac{\tilde{c}_\tau^\top \tilde{X}}{\sqrt{\tilde{c}_\tau^\top \tilde{c}_\tau}} \right)^2 = (v_\tau^\top X)^2.$$

Therefore, for each fixed τ , the likelihood-ratio test is equivalent to thresholding

$$|v_\tau^\top X|.$$

Taking the maximum over all candidate change-point locations $\tau \in [n-1]$, we obtain a test of the form

$$\max_{\tau \in [n-1]} |v_\tau^\top X| > \lambda$$

for some threshold λ . This has exactly the same structural form as the CUSUM classifier, except that the vectors v_τ are now determined by the generalized regression model rather than by the simple change-in-mean construction.

Hence, by the same neural-network construction as in Lemma 1, this test can be represented by a one-hidden-layer network with width $2n-2$. Therefore there exists

$$h^* \in \mathcal{H}_{1, 2n-2}$$

that is equivalent to the likelihood-ratio test for testing $\phi = 0$ against $\phi \neq 0$. $\square$

2.3 Generalisation Bound

In this section, we analyze the error rate and generalization bound of a neural network classifier for the single change-point problem.

Step 1: Generalization Bound. By standard VC generalization theory (Corollary 3.19 Bartlett et al. (2019)), for any $\delta > 0$, with probability at least $1 - \delta$, the empirical risk minimizer h_{ERM} satisfies

$$R(h_{\text{ERM}}) \leq L_N(h_{\text{ERM}}) + \sqrt{\frac{8d \log \frac{2eN}{d} + 8 \log(\frac{4}{\delta})}{N}},$$

where $R(h)$ is the population risk, $L_N(h)$ is the empirical risk, and $d = \text{VCdim}(\mathcal{H})$ is the VC dimension of the hypothesis class.

Step 2: Comparison with the CUSUM Classifier. Since h_{ERM} minimizes the empirical risk over $\mathcal{H}_{1,2n-2}$, we have

$$L_N(h_{\text{ERM}}) \leq L_N(h), \quad \forall h \in \mathcal{H}_{1,2n-2}.$$

In particular, since the CUSUM classifier $h_{\lambda}^{\text{CUSUM}} \in \mathcal{H}_{1,2n-2}$ (by Corollary 4.1), it follows that

$$L_N(h_{\text{ERM}}) \leq L_N(h_{\lambda}^{\text{CUSUM}}).$$

Moreover, Corollary 4.1 guarantees that

$$R(h_{\lambda}^{\text{CUSUM}}) \leq ne^{-nB^2/8}.$$

Step 3: Bounding the Complexity Term. By Theorem 7 (Bartlett et al., 2019), the VC dimension of the neural network class satisfies

$$d = \text{VCdim}(\mathcal{H}_{1,2n-2}) \leq CW \log W,$$

where W is the number of parameters. For a single hidden layer network with width $2n - 2$, we have $W = O(n^2)$, and hence

$$d = O(n^2 \log n).$$

Substituting this into the generalization bound yields

$$R(h_{\text{ERM}}) \leq ne^{-nB^2/8} + C \sqrt{\frac{n^2 \log n \log N + \log(1/\delta)}{N}}.$$

Therefore, the test error of the neural network classifier is bounded by the sum of the approximation error (given by the CUSUM classifier) and the estimation error (controlled by the VC dimension).

To begin with, we first discuss the error rate on the training set.

We write $P(n, \tau, \mu_L, \mu_R)$ for the distribution of the multivariate normal random vector $X \sim N_n(\mu, I_n)$ where

$$\mu := (\mu_L \mathbf{1}\{i \leq \tau\} + \mu_R \mathbf{1}\{i > \tau\})_{i \in [n]}.$$

Define $\eta := \tau/n$.

Lemma 3. Fix $\varepsilon \in (0, 1)$. Suppose $X \sim P(n, \tau, \mu_L, \mu_R)$ for some $\tau \in \mathbb{Z}^+$ and $\mu_L, \mu_R \in \mathbb{R}$.

1. If $\mu_L = \mu_R$, then

$$\mathbb{P}\left\{\|C(X)\|_{\infty} > \sqrt{2 \log(n/\varepsilon)}\right\} \leq \varepsilon.$$

2. If

$$|\mu_L - \mu_R| \sqrt{\eta(1-\eta)} > \sqrt{\frac{8 \log(n/\varepsilon)}{n}},$$

then

$$\mathbb{P}\left\{\|C(X)\|_{\infty} \leq \sqrt{2 \log(n/\varepsilon)}\right\} \leq \varepsilon.$$

Here,

$$|\mu_L - \mu_R| \sqrt{\tau(n-\tau)/n} = |\mu_L - \mu_R| \sqrt{\eta(1-\eta)}$$

can be interpreted as the signal-to-noise ratio of the mean change problem.

Proof. Since $X \sim N_n(\mu, I_n)$, and $v_i := \left(\sqrt{\frac{n-i}{in}} \mathbf{1}_i^\top, -\sqrt{\frac{i}{(n-i)n}} \mathbf{1}_{n-i}^\top \right)^\top$,

$$E(v_i^T X) = 0, \text{var}(v_i^T X) = i \frac{n-i}{in} + (n-i) \frac{i}{(n-i)n} = 1$$

Thus,

$$v_i^T X \sim N(0, 1).$$

According to the tail bound of Guassian, one has

$$P(|v_i^T X| > t) \leq e^{-\frac{t^2}{2}}.$$

Since

$$\begin{aligned} P(\sup |v_i^T X| \geq t) &= P(\cup_{i=1}^{n-1} \{v_i^T X \geq t\}) \\ &\leq \sum_{i=1}^{n-1} P(\{v_i^T X \geq t\}) \\ &\leq (n-1) e^{-\frac{t^2}{2}}. \end{aligned}$$

For simplicity, the author write

$$P(\|C(X)\|_\infty \geq t) \leq n e^{-\frac{t^2}{2}}.$$

Let $t = \sqrt{2 \log(\frac{n}{\epsilon})}$, one has

$$P\left(\|C(X)\|_\infty \geq \sqrt{2 \log(\frac{n}{\epsilon})}\right) \leq \epsilon.$$

In nature, it is the tail bound of the union of Guassian.

For the second part, it means that when the gap of change is large, the error is also small. The core of the proof is from the following observation.

Since $X = \mu + \xi$, one has

$$C(X) = C(\mu) + C(\xi).$$

Thus

$$\begin{aligned} \|C(X)\|_\infty &= \|C(\mu) + C(\xi)\|_\infty \\ &\geq \|C(\mu)\|_\infty - \|C(\xi)\|_\infty. \end{aligned}$$

It is noted that

$$\begin{aligned} \|C(\mu)\|_\infty &= |v_\tau \mu| \\ &= \left| \tau \sqrt{\frac{n-\tau}{\tau n}} \mu_L - (n-\tau) \sqrt{\frac{\tau}{(n-\tau)n}} \mu_R \right| \\ &= \sqrt{\frac{\tau(n-\tau)}{n}} |\mu_L - \mu_R|. \end{aligned}$$

and $\|C(\xi)\|_\infty$ is just the same with the previous proof, it is controled by the tail bound of the union Guassian. One has

$$P\left(\|C(Z)\|_\infty \leq \sqrt{2 \log\left(\frac{n}{\epsilon}\right)}\right) > 1 - \epsilon.$$

If

$$\sqrt{\frac{\tau(n-\tau)}{n}} |\mu_L - \mu_R| \geq \sqrt{8 \log\left(\frac{n}{\epsilon}\right)},$$

one has

$$\sqrt{\frac{\tau(n-\tau)}{n}} |\mu_L - \mu_R| - \sqrt{2 \log\left(\frac{n}{\epsilon}\right)} \geq \sqrt{2 \log\left(\frac{n}{\epsilon}\right)}.$$

It can be derived that

$$P\left(\|C(X)\|_\infty \geq \sqrt{2 \log\left(\frac{n}{\epsilon}\right)}\right) \geq 1 - \epsilon.$$

i.e.

$$P\left(\|C(X)\|_\infty \leq \sqrt{2\log\left(\frac{n}{\epsilon}\right)}\right) \leq \epsilon.$$

□

The lemma 3 shows that if there is no change point, then the probability that the CUSUM statistic exceeds the threshold can be controlled. In particular, if we choose the classification threshold

$$\lambda = \sqrt{2\log(n/\epsilon)},$$

then the probability of misclassification (false detection of a change point) is at most ϵ .

The second part shows that when a change point truly exists and the signal is sufficiently strong, the CUSUM statistic is very unlikely to miss the change.

For any $B > 0$, define

$$\Theta(B) := \left\{(\tau, \mu_L, \mu_R) \in [n-1] \times \mathbb{R} \times \mathbb{R} : |\mu_L - \mu_R| \sqrt{\tau(n-\tau)/n} \in \{0\} \cup (B, \infty)\right\}.$$

Thus $\Theta(B)$ is the parameter space of data distributions where there is either no change, or a single change-point in mean whose signal-to-noise ratio is at least B .

Corollary 1. Fix $B > 0$. Let π_0 be any prior distribution on $\Theta(B)$, then draw $(\tau, \mu_L, \mu_R) \sim \pi_0$ and $X \sim P(n, \tau, \mu_L, \mu_R)$, and define

$$Y = \mathbf{1}\{\mu_L \neq \mu_R\}.$$

For $\lambda = B\sqrt{n}/2$, the classifier h_λ^{CUSUM} satisfies

$$\mathbb{P}(h_\lambda^{\text{CUSUM}}(X) \neq Y) \leq ne^{-nB^2/8}.$$

Lemma3 provides probability bounds for the CUSUM statistic under the two possible scenarios: when there is no change point and when there is a change point with sufficiently large signal-to-noise ratio. In particular, it controls the probabilities of false positives and false negatives separately.

Corollary 1 combines these bounds and shows that the CUSUM-based classifier has a small overall misclassification risk over the parameter space $\Theta(B)$. Specifically, if the threshold is chosen as

$$\lambda = \frac{B\sqrt{n}}{2},$$

then the probability of misclassification satisfies

$$\mathbb{P}(h_\lambda^{\text{CUSUM}}(X) \neq Y) \leq ne^{-nB^2/8}.$$

Thus, while Lemma3 controls the behaviour of the CUSUM statistic, the corollary translates these bounds into a guarantee on the classification error of the CUSUM-based change-point detector.

Now we have analyzed the error rate of the CUSUM classifier.

Due to the fact that the CUSUM classifier is in the function space of $H_{L,m}$, the error rate on training set would be smaller than the CUSUM classifier, which gives us a very nice benchmark. We only need to analyze the complexity term. By Bartlett et al. (2019). (Theorem 7), the VC dimension of a neural network can be bounded by the number of layer and the neurons each layer.

theorem 1. Fix $B > 0$ and let π_0 be any prior distribution on $\Theta(B)$. We draw $(\tau, \mu_L, \mu_R) \sim \pi_0$, $X \sim P(n, \tau, \mu_L, \mu_R)$, and set

$$Y = \mathbf{1}\{\mu_L \neq \mu_R\}.$$

Suppose that the training data

$$\mathcal{D} := ((X^{(1)}, Y^{(1)}), \dots, (X^{(N)}, Y^{(N)}))$$

consist of independent copies of (X, Y) and

$$h_{\text{ERM}} := \arg \min_{h \in \mathcal{H}_{1,2n-2}} L_N(h)$$

is the empirical risk minimiser.

There exists a universal constant $C > 0$ such that for any $\delta \in (0, 1)$, with probability $1 - \delta$,

$$\mathbb{P}(h_{\text{ERM}}(X) \neq Y \mid \mathcal{D}) \leq ne^{-nB^2/8} + C \sqrt{\frac{n^2 \log(n) \log(N) + \log(1/\delta)}{N}}.$$

Proof.

$$R(h_{\text{ERM}}) \leq L_N(h_{\text{ERM}}) + \sqrt{\frac{8d \log \frac{2eN}{d} + 8 \log(\frac{4}{\delta})}{N}},$$

where $R(h)$ is the population risk, $L_N(h)$ is the empirical risk, and $d = \text{VCdim}(\mathcal{H})$ is the VC dimension of the hypothesis class.

Since h_{ERM} minimizes the empirical risk over $\mathcal{H}_{1,2n-2}$, we have

$$L_N(h_{\text{ERM}}) \leq L_N(h), \quad \forall h \in \mathcal{H}_{1,2n-2}.$$

In particular, since the CUSUM classifier $h_{\lambda}^{\text{CUSUM}} \in \mathcal{H}_{1,2n-2}$ (by Corollary 4.1), it follows that

$$L_N(h_{\text{ERM}}) \leq L_N(h_{\lambda}^{\text{CUSUM}}).$$

Moreover, Corollary 3 guarantees that

$$R(h_{\lambda}^{\text{CUSUM}}) \leq ne^{-nB^2/8}.$$

On the other hand, Theorem A.5 states that the VC dimension of a fully connected neural network satisfies

$$\text{VCdim}(\mathcal{F}) = O(\bar{L} W \log(pU)),$$

where W is the total number of parameters, $\bar{L}$ is the effective depth of the network, U is the total number of computational units, p is the number of pieces in the piecewise polynomial activation function, and d is the degree of each polynomial piece.

Let m_i denote the number of units in layer i , with $m_0 = n$ the input dimension. For each neuron in the i -th layer, there are m_{i-1} neurons in the $i-1$ -th layer to connect and one bias term b . Thus the total number of parameters is

$$W = \sum_{i=1}^L m_i(m_{i-1} + 1).$$

Define the cumulative number of parameters up to layer i as

$$W_i = \sum_{j=1}^i m_j(m_{j-1} + 1).$$

The effective depth is then given by

$$\bar{L} = \frac{1}{W} \sum_{i=1}^L W_i = \frac{1}{W} \sum_{j=1}^L (L - j + 1) m_j(m_{j-1} + 1).$$

Finally, let

$$U = \sum_{i=1}^L m_i$$

be the total number of computational units in the network.

This result shows that the complexity of neural networks is primarily determined by the number of parameters W and how these parameters are distributed across layers, as captured by the effective depth $\bar{L}$. The logarithmic term $\log(pU)$ reflects additional contributions from the architecture, such as the number of units and the complexity of activation functions.

For $H_{1,2n-2}$,

$$\begin{aligned} L &= 1, \\ m_0 &= 2n - 2, \\ W &= n(2n - 1) + (n + 1) = O(n^2), \\ U &= n + (2n - 2) = O(n). \end{aligned}$$

Thus,

$$d = O(n^2 \log n)$$

Thus

$$\sqrt{\frac{8d \log \frac{2eN}{d} + 8 \log(\frac{4}{\delta})}{N}} \leq C \sqrt{\frac{8n^2 \log n \log \frac{N}{n^2 \log n} + 8 \log(\frac{4}{\delta})}{N}}$$

Using $\log(N/(n^2 \log n)) \leq \log N$, we further obtain

$$\sqrt{\frac{8d \log(2eN/d) + 8 \log(4/\delta)}{N}} \leq C \sqrt{\frac{n^2 \log n \log N + \log(1/\delta)}{N}}.$$

□

Theorem 1 shows that the empirical risk minimiser within the neural network class $\mathcal{H}_{1,2n-2}$ has good generalisation performance for the change-point detection problem over the parameter space $\Theta(B)$.

In particular, the misclassification error of the learned classifier is bounded by two terms. The first term,

$$ne^{-nB^2/8},$$

is the intrinsic error of the CUSUM-based classifier, as shown in Corollary 4.1. The second term depends on the complexity of the neural network class and decreases as the training sample size N increases.

Therefore, when N is sufficiently large (e.g. $N \gg n^2 \log n$), the empirical risk minimiser in $\mathcal{H}_{1,2n-2}$ achieves performance comparable to the optimal CUSUM-based classifier.

The next Theorem 2 extends the result of Theorem 1 from a one-hidden-layer neural network to a deep architecture. And it shows the result that even we don't use all the point in the time series, we can still do well.

The procedure is similar. First use CUSUM classifier as benchmark to get an estimate for the error on training set. Then use the VC dimension to control the complexity term.

But the outcome is tricky. Because the extension from one layer to multilayer and from entire time series to a fraction seem to be separate. The previous is controlled by the benchmark, CUSUM, and the latter is controlled by VC dimension.

Similarly, we first introduce one lemma and one corollary to show the benchmark in CUSUM classifier but not use the entire time series.

Here, we let $T_0 \subset [n-1]$ be a subset of candidate change-point locations such that

$$|T_0| = 2 \lfloor \log_2 n \rfloor.$$

We use T_0 instead of the full set $[n-1]$ to reduce the complexity of the problem.

Lemma 4. Fix $\epsilon \in (0, 1)$ and suppose that

$$X \sim P(n, \tau, \mu_L, \mu_R)$$

for some $\tau \in [n-1]$ and $\mu_L, \mu_R \in \mathbb{R}$.

(a) If $\mu_L = \mu_R$, then

$$\mathbb{P} \left\{ \max_{t \in T_0} |v_t^\top X| > \sqrt{2 \log \left(\frac{|T_0|}{\epsilon} \right)} \right\} \leq \epsilon.$$

(b) If

$$|\mu_L - \mu_R| \sqrt{\eta(1-\eta)} > \sqrt{\frac{24 \log(|T_0|/\epsilon)}{n}},$$

then

$$\mathbb{P} \left\{ \max_{t \in T_0} |v_t^\top X| \leq \sqrt{2 \log \left(\frac{|T_0|}{\epsilon} \right)} \right\} \leq \epsilon.$$

Proof. (a) For each $t \in [n-1]$, since $\|v_t\|_2 = 1$, we have

$$v_t^\top X \sim N(0, 1).$$

Hence, by the Gaussian tail bound and a union bound,

$$\mathbb{P}\left\{\max_{t \in T_0} |v_t^\top X| > y\right\} \leq \sum_{t \in T_0} \mathbb{P}(|v_t^\top X| > y) \leq |T_0|e^{-y^2/2}.$$

Taking

$$y = \sqrt{2 \log\left(\frac{|T_0|}{\epsilon}\right)}$$

proves part (a).

(b) There exists some $t_0 \in T_0$ such that

$$|t_0 - \tau| \leq \min\{\tau, n - \tau\}/2.$$

By Lemma 3, we have

$$|v_{t_0}^\top \mathbb{E}X| \geq \frac{\sqrt{3}}{3} \|\mathcal{C}(\mathbb{E}X)\|_\infty \geq \frac{\sqrt{3}}{3} |\mu_L - \mu_R| \sqrt{n\eta(1-\eta)} \geq 2\sqrt{2 \log\left(\frac{|T_0|}{\epsilon}\right)}.$$

Using the triangle inequality together with part (a), we obtain with probability at least $1 - \epsilon$ that

$$\max_{t \in T_0} |v_t^\top X| \geq |v_{t_0}^\top X| \geq |v_{t_0}^\top \mathbb{E}X| - |v_{t_0}^\top (X - \mathbb{E}X)| \geq \sqrt{2 \log\left(\frac{|T_0|}{\epsilon}\right)}.$$

This proves part (b). $\square$

Although replacing $[n - 1]$ by T_0 reduces the union bound and leads to a smaller threshold, it does not automatically imply a lower overall error. Instead, it reflects a trade-off between reducing false positives and maintaining sufficient sensitivity to detect the change-point.

Similarly, we use $\Theta(B)$ to uniform the results.

Corollary 2. Fix $B > 0$. Let π_0 be any prior distribution on $\Theta(B)$, then draw

$$(\tau, \mu_L, \mu_R) \sim \pi_0, \quad X \sim P(n, \tau, \mu_L, \mu_R),$$

and define

$$Y = \mathbf{1}\{\mu_L \neq \mu_R\}.$$

Then for

$$\lambda^* = \frac{B\sqrt{3n}}{6},$$

the test $h_{\lambda^*}^{\text{CUSUM*}}$ satisfies

$$\mathbb{P}(h_{\lambda^*}^{\text{CUSUM*}}(X) \neq Y) \leq 2\lfloor \log_2(n) \rfloor e^{-nB^2/24}.$$

Proof. Setting

$$\epsilon = |T_0|e^{-nB^2/24}$$

in Lemma 4, we have for any $(\tau, \mu_L, \mu_R) \in \Theta(B)$ that

$$\mathbb{P}\left(h_{\lambda^*}^{\text{CUSUM*}}(X) \neq \mathbf{1}\{\mu_L \neq \mu_R\}\right) \leq |T_0|e^{-nB^2/24}.$$

The conclusion follows by integrating over π_0 and using

$$|T_0| = 2\lfloor \log_2(n) \rfloor.$$

$\square$

theorem 2. Fix $B > 0$ and let the training data $\mathcal{D}$ be generated as in Theorem 1. Let

$$h_{\text{ERM}} := \arg \min_{h \in \mathcal{H}_{L,m}} L_N(h)$$

be the empirical risk minimiser for a neural network with $L \geq 1$ layers and $m = (m_1, \dots, m_L)^\top$ hidden layer widths.

If $m_1 > 4 \log_2(n)$ and $m_r m_{r+1} = O(n \log n)$ for all $r \in [L - 1]$, then there exists a universal constant $C > 0$ such that for any $\delta \in (0, 1)$, with probability $1 - \delta$,

$$\mathbb{P}(h_{\text{ERM}}(X) \neq Y \mid \mathcal{D}) \leq 2\lfloor \log_2(n) \rfloor e^{-nB^2/24} + C\sqrt{\frac{L^2 n \log^2(Ln) \log(N) + \log(1/\delta)}{N}}.$$

Proof. By the VC generalization bound, with probability at least $1 - \delta$,

$$R(h_{\text{ERM}}) \leq \min_{h \in \mathcal{H}_{L,m}} R(h) + \sqrt{\frac{8d \log \frac{2eN}{d} + 8 \log(\frac{4}{\delta})}{N}},$$

where $R(h)$ is the population risk, $d = \text{VCdim}(\mathcal{H}_{L,m})$, and N is the training sample size.

Since

$$h_{\lambda^*}^{\text{CUSUM}*} \in \mathcal{H}_{1, 4\lfloor \log_2 n \rfloor}.$$

and from lemma 4,

$$R(h_{\lambda^*}^{\text{CUSUM}*}) \leq 2\lfloor \log_2 n \rfloor e^{-nB^2/24}.$$

It remains to control the VC dimension d . By Theorem 5 (Bartlett et al. (2019)), the VC dimension of a fully connected neural network satisfies

$$\text{VCdim}(\mathcal{F}) = O(\bar{L} W \log(pU)).$$

Using the crude bound $\bar{L} \leq L$ and absorbing p into the constant, this yields

$$d = O(LW \log U).$$

Let m_r denote the width of the r -th hidden layer, with input dimension $m_0 = n$ and output dimension $m_{L+1} = 1$. Then the total number of parameters is

$$W^* = \sum_{r=0}^L m_{r+1}(m_r + 1).$$

Under the assumptions of Theorem 2,

$$m_1 \geq 4\lfloor \log_2 n \rfloor, \quad m_r m_{r+1} = O(n \log n), \quad r \in [L-1].$$

Hence each inter-layer parameter block is of order $O(n \log n)$, and therefore

$$W^* = O(Ln \log n).$$

Also, the total number of units is at most of the same logarithmic order in Ln , so we obtain

$$d = O(LW^* \log W^*) = O(L \cdot Ln \log n \cdot \log(Ln \log n)).$$

Since $\log n \leq \log(Ln)$ and $\log(Ln \log n) \leq C \log(Ln)$ for some universal constant C , this simplifies to

$$d = O(L^2 n \log^2(Ln)).$$

Substituting this bound into the generalization inequality gives

$$\sqrt{\frac{8d \log \frac{2eN}{d} + 8 \log(\frac{4}{\delta})}{N}} \leq C \sqrt{\frac{L^2 n \log^2(Ln) \log N + \log(1/\delta)}{N}},$$

where we used the crude bound

$$\log\left(\frac{2eN}{d}\right) \leq C \log N.$$

Combining the approximation term and the estimation term, we conclude that with probability at least $1 - \delta$,

$$R(h_{\text{ERM}}) \leq 2\lfloor \log_2 n \rfloor e^{-nB^2/24} + C \sqrt{\frac{L^2 n \log^2(Ln) \log N + \log(1/\delta)}{N}}.$$

This is exactly the desired result. □

□

Theorem 2 extends the result of Theorem 1 from single-hidden-layer networks to neural networks with multiple hidden layers and with less data to input. It shows that the empirical risk minimiser in the neural network class $\mathcal{H}_{L,m}$ achieves a small misclassification error in the change-point detection problem.

The error bound consists of two terms. The first term,

$$2\lfloor \log_2(n) \rfloor e^{-nB^2/24},$$

represents the intrinsic error of a modified CUSUM-based classifier. The second term depends on the complexity of the neural network and decreases as the training sample size N increases.

Therefore, even when the neural network has multiple hidden layers, and not all the time series as input, its classification performance can be comparable to that of the CUSUM-based change-point detector when the training sample size is sufficiently large.

3 Simulation in Paper

In the primary simulation study, the authors evaluate neural network–based change-point detection in a binary classification setting, where the task is to determine whether a single change-point in the mean is present in a time series.

The data are generated from a piecewise-constant mean model of length $n = 100$:

$$x_t = \mu_L \mathbf{1}(t \leq \tau) + \mu_R \mathbf{1}(t > \tau) + \varepsilon_t,$$

where τ is a randomly selected change-point location. Half of the generated sequences contain a change-point ($\mu_L \neq \mu_R$), and the other half contain no change ($\mu_L = \mu_R$). The training sample size varies from $N = 100$ to larger values, while the test set consists of 30,000 independently generated sequences.

To examine robustness under model misspecification, four noise scenarios are considered:

- **S1:** Independent Gaussian noise.
- **S1':** Autoregressive noise with fixed autocorrelation ($\rho = 0.7$).
- **S2:** Random autocorrelation structure with Gaussian innovations.
- **S3:** Heavy-tailed noise generated from a Cauchy distribution.

The neural networks used in this simulation are fully connected ReLU networks (multilayer perceptrons). The authors compare:

- A single-hidden-layer network with width $m = 4\lfloor \log_2(n) \rfloor$,
- A single-hidden-layer network with width $m = 2n - 2$ (sufficient to represent CUSUM exactly),
- Deeper networks with 5 and 10 hidden layers.

Table 1: Simulation 1: Relative Performance of Neural Network (NN) vs CUSUM

Noise Scenario	Relative Performance
Independent Gaussian	NN $\approx$ CUSUM
Autoregressive Noise	NN $>$ CUSUM
Random Dependence Structure	NN $>$ CUSUM
Heavy-tailed (Cauchy) Noise	NN $\gg$ CUSUM

4 Experiments

For piecewise linear networks with parameter number p :

$$VC = O(p \log p)$$

So,

$$Generalization\ Gap \propto \sqrt{\frac{p}{N}}$$

We wanna use SGD to train our NN. To examine whether deeper or overparameterized neural networks exhibit implicit regularization effects in change-point detection, we propose the following controlled simulation studies based on the binary change-point setting described in the paper.

4.1 Simulation Settings

Capacity Scaling Study (Width and Depth) We fix the training sample size (e.g., $N \in \{200, 500, 1000\}$) and systematically vary network capacity:

- **Width scaling:** single-hidden-layer networks with $m \in \{8, 16, 32, 64, 128, 256, 512\}$.
- **Depth scaling:** networks with $L \in \{1, 2, 5, 10, 20\}$ hidden layers (fixed width).

All models will be trained to near-zero training error when possible. We will record training error, test error, and the generalization gap. Stability of test error under increasing parameter count would provide evidence of implicit regularization beyond classical VC-dimension predictions.

Label Noise Stress Test To investigate memorization dynamics, we introduce label noise by randomly flipping a proportion $\eta \in \{0, 0.1, 0.2, 0.4\}$ of training labels. We will monitor:

- Training and test error as a function of epochs,
- The onset of overfitting behavior,
- The ability of deeper models to initially capture structural signal before memorizing noise.

If models first learn the underlying structure before fitting noise, this would indicate optimization-induced implicit bias.

Optimization-Induced Regularization (Batch Size and Learning Rate) Since implicit regularization is closely linked to optimization dynamics, we vary:

- Batch size $\in \{16, 32, 128, 512\}$,
- Learning rate $\in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$.

We compare test error under different configurations while achieving comparable training accuracy. Improved generalization under small batch sizes or larger learning rates would support the hypothesis that stochastic optimization induces implicit regularization.

4.2 Simulation Results

In the original paper:

- under Gaussian noise, neural networks are competitive with CUSUM,
- under autocorrelated or heavy-tailed noise, neural networks often outperform CUSUM.

In our current reproduction:

- the CUSUM baseline is often much stronger than the neural networks,
- some neural network settings exhibit test errors close to 0.5,
- the gap between our curves and the paper’s reported curves remains substantial.

This suggests that some implementation details in the original training pipeline may still not be fully recovered.

Experiment 1: Depth Extension Under Original Simulation Setting

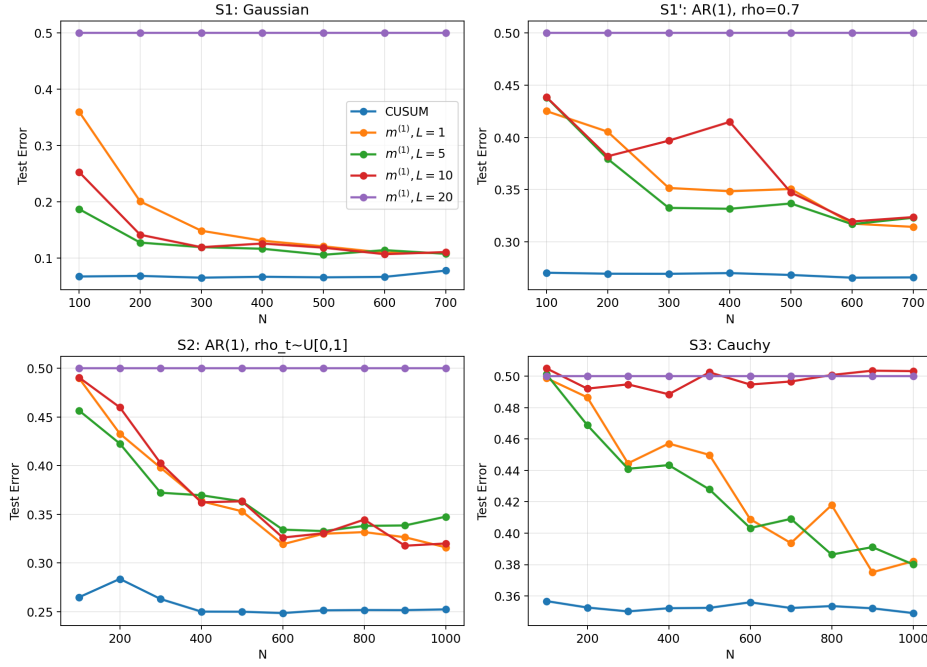


Figure 2: Our Results

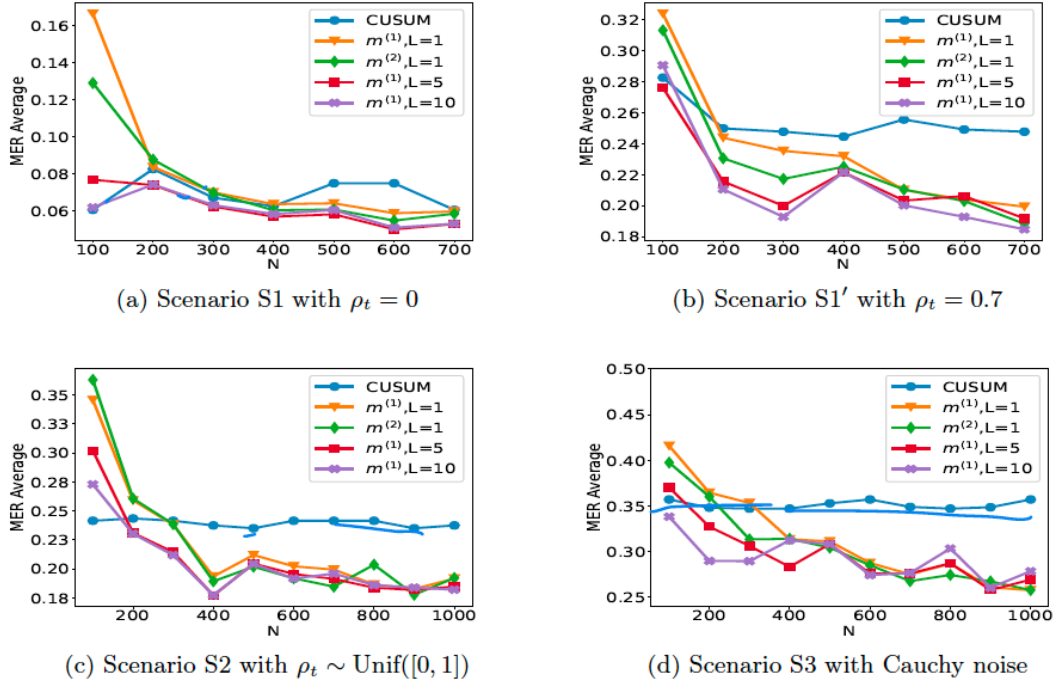


Figure 3: Original Paper Results

Experiment 2: Label Noise Stress Test We fixed $N = 500$ and randomly flipped a proportion η of training labels.

Experiment 2: Label Noise Stress Test (Fixed $N=500$)

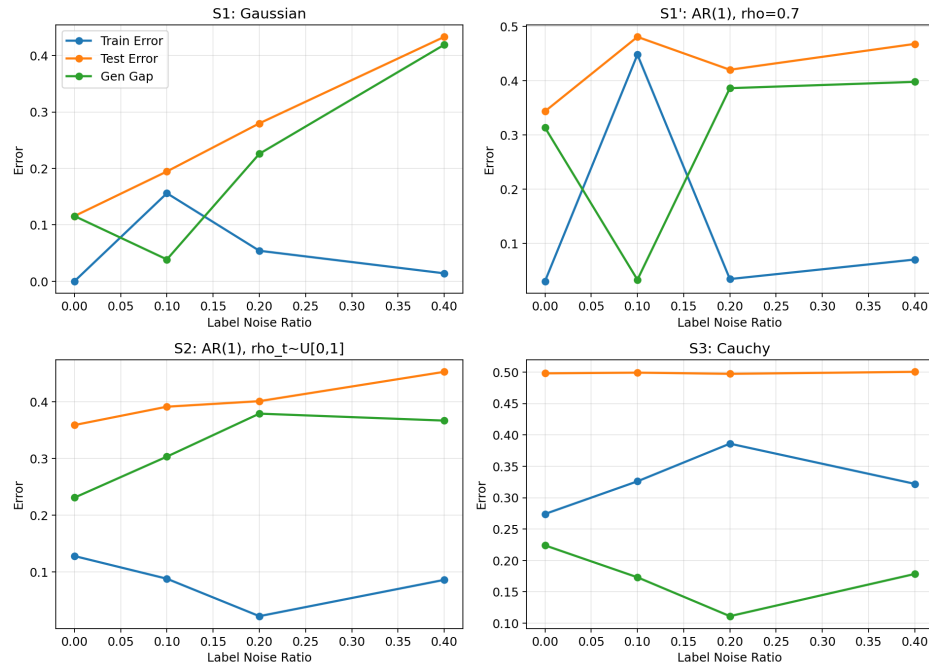


Figure 4: Label Noise Stress

Experiment 3: Learning Rate and Batch Size Each panel shows the estimation error as a function of the learning rate and batch size under different data-generating distributions. The results demonstrate that the optimization landscape and stability of training depend strongly on the underlying data distribution, with heavier-tailed designs (e.g., Cauchy) exhibiting larger sensitivity to hyperparameter choices.

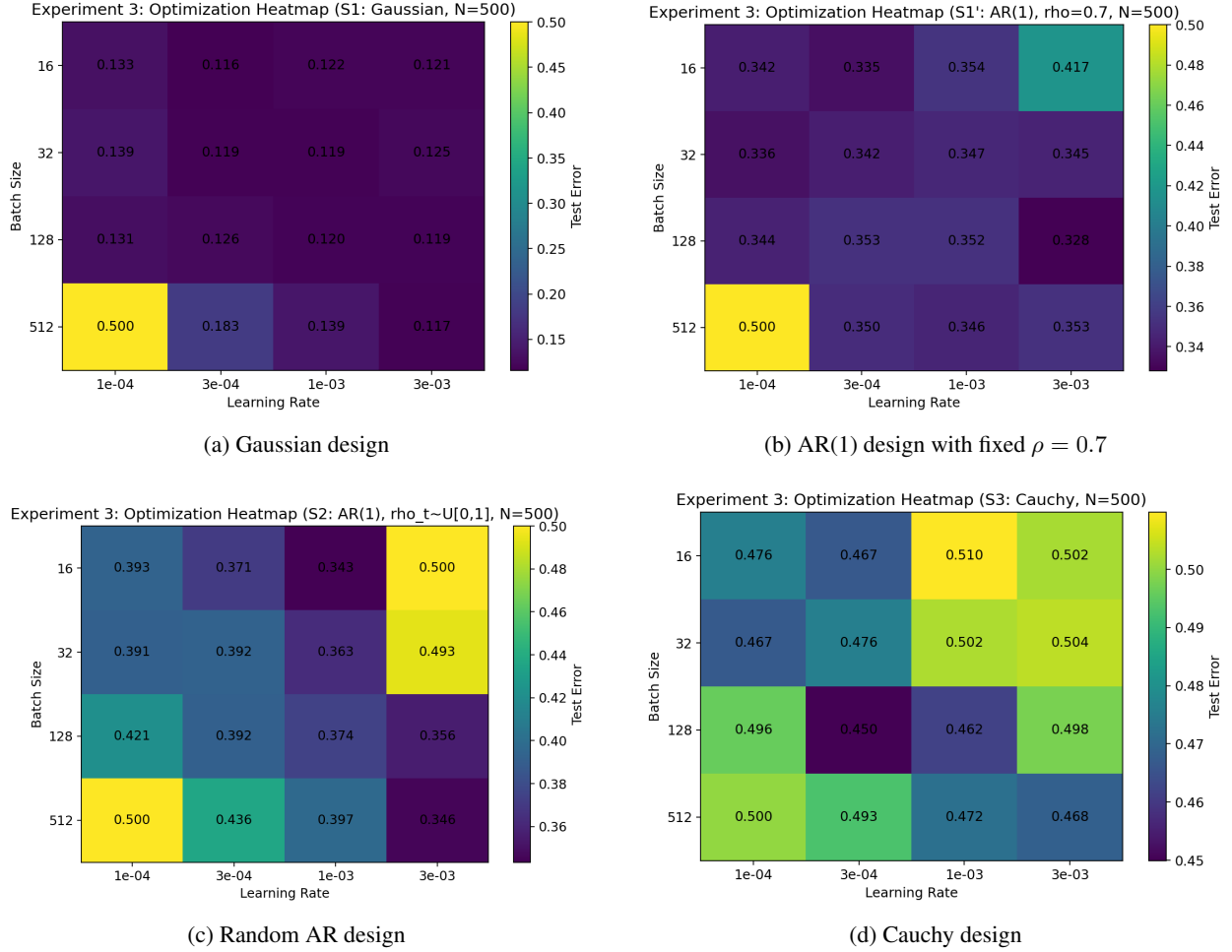


Figure 5: Optimization heatmaps for Experiment 3 with $N = 500$.

5 Response to Questions

5.1 Specific rates in the theorems you mentioned come from

5.2 Comments on the hyperparameter choices.

Following the paper, the neural-network training hyperparameters in the simulation study are deliberately kept simple and fixed across all scenarios: each input sequence is first (min-max)rescaled to $[0, 1]$, and the networks are then trained using the cross-entropy loss, the Adam optimizer, learning rate 0.001, batch size 32, and 200 epochs.

The paper explicitly states that these training hyperparameters were selected “based on a training dataset with cross-validation”, with additional details deferred to Appendix B (Their paper), only gave the fixed hyperparameters but not the reasons. In the same “spirit”, the threshold of the CUSUM classifier is also tuned using the training data.

Theory-motivated architectural choices. A more interesting aspect of the paper is that some of the architectural hyperparameters are motivated by the theoretical results developed earlier in the paper. In particular, the four neural-network classes compared in the main simulation are

$$H_{1,m^{(1)}}, \quad H_{1,m^{(2)}}, \quad H_{5,m^{(1)}1_5}, \quad H_{10,m^{(1)}1_{10}},$$

where

$$m^{(1)} = 4\lfloor \log_2(n) \rfloor, \quad m^{(2)} = 2n - 2.$$

The width $m^{(2)} = 2n - 2$ comes from the exact single-hidden-layer neural-network representation of the CUSUM-based classifier in Lemma 3.1, while the smaller width $m^{(1)} = 4\lfloor \log_2(n) \rfloor$ is tied to the pruned representation used in Theorem 4.3. Hence, these widths are not arbitrary: they are directly connected to the theoretical constructions showing that CUSUM-type classifiers can be represented within the neural-network classes under study.

Theory-motivated signal scaling. The simulation design also contains a theory-driven choice in the signal magnitude. The post-change mean μ_R is generated through a scale factor

$$b = \sqrt{\frac{8n \log(20n)}{\tau(n - \tau)}},$$

and the paper explains that this choice is made in line with Lemma 4.1, so that the generated signals cover a meaningful range of signal-to-noise ratios. Thus, the simulation does not simply use an arbitrary fixed jump size; instead, the jump magnitude depends on the change-point location τ in a way that is consistent with the theoretical detection boundary discussed earlier in the paper.

Interpretation. Overall, the paper combines two types of hyperparameter choices. First, the **optimization hyperparameters** (learning rate, batch size, number of epochs, optimizer) are chosen pragmatically by "cross-validation" (which is hidden and not showed up in the entire appendix) and then fixed across all experiments.

Second, the **architectural and signal-scale hyperparameters** are partly theory-guided: the network widths $m^{(1)}$ and $m^{(2)}$ are derived from representation and generalization results, and the scaling factor b is tied to the theoretical signal-to-noise boundary. This is an important feature of the simulation study, because it means that the experimental setup is not purely empirical; instead, it is designed to mirror the theoretical development in the paper as closely as possible.

References

- Bartlett, P. L., Harvey, N., Liaw, C., and Mehrabian, A. (2019). Nearly-tight vc-dimension and pseudodimension bounds for piecewise linear neural networks. *Journal of Machine Learning Research*, 20(63):1–17.
- Hadash, G., Kermany, E., Carmeli, B., Lavi, O., Kour, G., and Jacovi, A. (2018). Estimate and replace: A novel approach to integrating deep neural networks with existing applications. *arXiv preprint arXiv:1804.09028*.
- Kour, G. and Saabne, R. (2014a). Fast classification of handwritten on-line arabic characters. In *Soft Computing and Pattern Recognition (SoCPaR), 2014 6th International Conference of*, pages 312–318. IEEE.
- Kour, G. and Saabne, R. (2014b). Real-time segmentation of on-line handwritten arabic script. In *Frontiers in Handwriting Recognition (ICFHR), 2014 14th International Conference on*, pages 417–422. IEEE.
- Mohri, M., Rostamizadeh, A., and Talwalkar, A. (2012). *Foundations of Machine Learning*. Adaptive Computation and Machine Learning Series. MIT Press, Cambridge, MA.

A Auxiliary Results

In this appendix, we collect several auxiliary results the author used in the proof of main theorems.

Lemma 5 (Neural network VC-dimension bound Bartlett et al. (2019)). *Consider a neural network architecture with W parameters and U computation units arranged in L layers, so that each unit has connections only from units in earlier layers. Let k_i denote the number of units at the i th layer. Suppose that all non-output units have piecewise-polynomial activation functions with $p + 1$ pieces and degree no more than d , and the output unit has the identity function as its activation function.*

If $d = 0$, let W_i denote the number of parameters (weights and biases) at the inputs to units in layer i ; if $d > 0$, let W_i denote the total number of parameters (weights and biases) at the inputs to units in all the layers up to layer i (i.e., in layers $1, 2, \dots, i$). Define the effective depth as

$$\bar{L} := \frac{1}{W} \sum_{i=1}^L W_i,$$

and let

$$R := \sum_{i=1}^L k_i (1 + (i-1)d^{i-1}) \leq U + U(L-1)d^{L-1}.$$

For the class $\mathcal{F}$ of all (real-valued) functions computed by this network and $m \geq \bar{L}W$, we have

$$\Pi_{\text{sgn}(\mathcal{F})}(m) \leq \prod_{i=1}^L 2 \left(\frac{2emk_i p (1 + (i-1)d^{i-1})}{W_i} \right)^{W_i}.$$

If $U > 2$, then

$$\text{VCdim}(\mathcal{F}) \leq L + \bar{L}W \log_2(4epR \log_2(2epR)) = O(\bar{L}W \log(pU) + \bar{L}LW \log d).$$

In particular, if $d = 0$, then

$$\text{VCdim}(\mathcal{F}) \leq L + W \log_2(4epU \log_2(2epU)) = O(W \log(pU)).$$

If $d = 1$, then

$$\text{VCdim}(\mathcal{F}) \leq L + \bar{L}W \log_2 \left(4ep \sum_{i=1}^L ik_i \log_2 \left(\sum_{i=1}^L 2epik_i \right) \right) = O(\bar{L}W \log(pU)).$$

Lemma 6 (VC-dimension generalization bound Mohri et al. (2012)). *Let $\mathcal{H}$ be a family of functions taking values in $\{-1, +1\}$ with VC-dimension d . Then for any $\delta > 0$, with probability at least $1 - \delta$, the following holds for all $h \in \mathcal{H}$:*

$$R(h) \leq \hat{R}_S(h) + \sqrt{\frac{2d \log(\frac{em}{d})}{m}} + \sqrt{\frac{\log(\frac{1}{\delta})}{2m}}.$$